

Containers Module Constitution

INHERITED FROM the Helix Constitution

This module is governed by the Helix Constitution. All rules in the constitutionâ€™s `CONSTITUTION.md` and the `Constitution.md` it references apply unconditionally. Locate the constitution from any nested depth via its `find_constitution.sh` helper â€” do NOT hardcode a path (this module stays fully decoupled and project-agnostic per Â§11.4.28).

Canonical reference: <https://github.com/HelixDevelopment/HelixConstitution>

Scope

`digital.vasic.containers` is a generic, project-agnostic Go module for container orchestration, health checking, lifecycle management, remote distribution, and service discovery across Docker, Podman, and Kubernetes runtimes. It is the single integration point through which the consuming projectâ€™s binary (and any other consumer) brings up its full container topology â€” local and remote â€” driven entirely by the consumerâ€™s `.env` file (`Containers/.env` for the consuming project).

This module is foundational: it has no upstream sibling modules and is consumed by `Challenges`, `HelixLLM`, `HelixQA`, and the consuming project itself.

Module-Specific Invariants

1. **Project-agnostic.** No hardcoded project-specific package names, endpoints, device serials, or fixtures. All consumer-specific data is registered via the public API. Default values are empty or generic.
2. **Sole orchestrator role.** The moduleâ€™s runtime is the only sanctioned path for container lifecycle operations. Direct `docker/podman` commands and `docker-compose up|down` are prohibited as workflows in any consumer.
3. **Dynamic remote-host enrolment (CONST-031).** Remote hosts are loaded from `CONTAINERS_REMOTE_HOST_N_*` env vars ($N=1..100$). The loader (`pkg/envconfig/parser.go`) stops at the first absent `_NAME`. No hostname is hardcoded anywhere else in the repo.
4. **Rootless / no sudo.** No `sudo` or `su` in source, scripts, tests, or docs. Use rootless container runtimes only.
5. **Health-check parity.** Every service started by this module must expose a TCP or HTTP health endpoint and pass `HealthChecker` checks with retries before being considered up.
6. **Rebuild-on-change.** Any code change affecting a containerised component requires rebuilding and redeploying the container locally, and re-running with `CONTAINERS_REMOTE_ENABLED=true` for remote distribution.